

Software Requirements Specification (SRS)

Project Title: Emergency Resource & Disaster Relief Dispatch Coordinator

Revision Note: This is a corrected version addressing syllabus-compliance gaps identified during review (see attached changelog). New/modified content is marked in the source discussion; this document itself reads as the final clean SRS.

1. Introduction

1.1 Purpose

This document specifies the software requirements for the "Emergency Resource & Disaster Relief Dispatch Coordinator." It defines the functional and non-functional requirements, system architecture, testing approach, and technical constraints necessary to guide the development, testing, and deployment phases of the capstone project.

1.2 Scope

The system is a centralized, distributed web application designed to manage localized disaster relief efforts. It bridges the gap between stranded citizens, field volunteers, and disaster management administrators. The software facilitates real-time SOS requests, SMS-based offline emergency ingestions, live inventory tracking of rescue supplies, dispatch workflows, and external donations.

1.3 Technology Stack Constraints

The project must strictly adhere to the mandatory Java Full-Stack curriculum requirements:

- Frontend: React.js utilizing Axios or Fetch for REST API integration.
- Backend: Java Spring Boot REST API.
- Database: MySQL utilizing Spring Data JPA/Hibernate for Object-Relational Mapping (ORM).
- Security: Stateless JWT (JSON Web Tokens) authentication with Role-Based Access Control. The signed JWT shall be stored securely on the client (localStorage or sessionStorage) and attached to subsequent requests via the Authorization header.
- API Testing: Postman shall be used to design, execute, and document test collections for all REST endpoints.
- Version Control: Git, with the source repository hosted on GitHub.
- Local/Test Database: An in-memory H2 database shall be used for local unit and integration testing, while MySQL remains the database of record for development and production environments.
- Deployment: Containerized deployment using Docker and Docker Compose, with final hosting on a cloud platform (AWS EC2 or Heroku).

[Added: Postman-based API Testing, Git/GitHub version control, cloud hosting target, JWT client-storage detail, and H2 for local testing — previously missing from the stack constraints.]

1.4 Intended Audience

This document is intended for academic evaluation committees, internal faculty supervisors, and the student development team (acting as frontend, backend, and database engineers).

2. Overall Description

2.1 Product Perspective

The system is a standalone, web-based platform. It operates over a standard HTTP network but integrates a third-party SMS gateway (e.g., Twilio) via a webhook, allowing the backend to process emergency requests originating from standard cellular networks independently of civilian internet access.

2.2 User Classes and Characteristics

The system strictly segregates users via JWT RBAC into four distinct classes:

- **Administrator (Disaster Cell):** Has absolute system authority. Manages inventory, oversees triage maps, and assigns tasks to volunteers.
- **Volunteer (Field Worker):** Requires mobile-responsive access. Receives assigned dispatch tasks, logs delivery statuses, and updates field conditions.
- **Citizen (Victim/End-User):** Requires a frictionless, low-bandwidth UI. Submits SOS requests and views their status.
- **Donor (External Contributor):** Can access public-facing donation pages to pledge funds or physical supplies.

2.3 Operating Environment

- **Server Environment:** Linux-based Docker containers orchestrating a Spring Boot embedded Tomcat server (port 8080) and a MySQL 8.0 instance (port 3306), hosted on a cloud platform (AWS EC2 / Heroku) for final deployment.
- **Client Environment:** Modern web browsers (Chrome, Firefox, Safari) executing a React.js Single Page Application (SPA).

[Modified: Server Environment now explicitly names the cloud hosting target, consistent with the syllabus's LO6 requirement.]

3. Functional Requirements

3.1 Module 1: Identity & Access Management (IAM)

- FR-1.1: The system shall provide endpoints for user registration and login (/api/v1/auth/register, /api/v1/auth/login).
- FR-1.2: The backend shall generate and return a signed JWT upon successful authentication.
- FR-1.3: The system shall extract user roles from the JWT payload to restrict or grant access to specific endpoints.

3.2 Module 2: Citizen SOS & Triage Engine

- FR-2.1: Citizens shall be able to submit a web-based SOS form including geolocation coordinates, urgency level (High, Medium, Low), and required supplies.

- FR-2.2 (Offline Fallback): The Spring Boot backend shall expose an unprotected POST `/api/v1/sos/webhook/sms` endpoint to ingest x-www-form-urlencoded payloads from an external SMS provider.
- FR-2.3: The system shall parse incoming SMS text strings to extract sender phone numbers and location keywords, automatically generating a "Pending" database record.
- FR-2.4: The system shall allow Administrators to filter and sort SOS requests by urgency level, status, and submission time, using paginated Spring Data JPA query methods (e.g., `findByUrgencyLevel()`, `findByStatus()`).

[Added FR-2.4: Filtering, sorting, and pagination of SOS requests — maps to the syllabus's explicit requirement for custom JPA query methods and pagination/sorting (Module IV, LO4).]

3.3 Module 3: Live Inventory Management

- FR-3.1: Administrators shall be able to execute full CRUD operations (Create, Read, Update, Delete) on the relief supplies database via the React frontend.
- FR-3.2: The system shall use Hibernate ORM to automatically deduct stock quantities when items are assigned to an active dispatch task.
- FR-3.3: The dashboard shall display a low-stock alert when an item's quantity falls below a predefined `minimum_threshold`.
- FR-3.4: The system shall support paginated retrieval of inventory records, and allow querying by category or `minimum_threshold` breach, using Spring Data JPA's Pageable and custom query methods.

[Added FR-3.4: Pagination and custom query support for inventory listing — same syllabus mapping as above.]

3.4 Module 4: Volunteer Dispatch & Workflow

- FR-4.1: Administrators shall link a specific `volunteer_id` to a `request_id` to initiate a dispatch task.
- FR-4.2: Volunteers shall be able to update their assigned task status sequentially (e.g., ASSIGNED → EN_ROUTE → DELIVERED).
- FR-4.3: The system shall log a timestamp for every status change.

3.5 Module 5: Donation Portal

- FR-5.1: Donors shall be able to submit pledges for physical goods, which admins can approve to increment MySQL inventory tables.
- FR-5.2: The system shall integrate a sandbox payment gateway (e.g., Razorpay Test Mode) to simulate monetary donations, recording transaction IDs in the database.

3.6 Module 6: API Testing & Quality Assurance

- FR-6.1: Every REST endpoint defined in Modules 1–5 shall have a corresponding Postman collection covering at least one success-path and one failure-path (e.g., invalid token, missing field) test case.
- FR-6.2: Postman collections shall be version-controlled alongside the source code and exported as part of the final project documentation.
- FR-6.3: Environment variables (e.g., base URL, JWT token) shall be configured in Postman to support testing across local and deployed (cloud) environments.

[Added new Module 3.6 in its entirety: API Testing & QA. The syllabus explicitly lists Postman-based API testing as a required experiment and as a named Capstone deliverable ("Testing: API testing with Postman"); the original SRS had no functional requirement covering this at all.]

4. External Interface Requirements

4.1 User Interfaces (UI)

- The frontend shall be built using React.js.
- The UI must implement responsive design using Bootstrap, ensuring the Volunteer and Citizen dashboards are fully functional on mobile devices.
- The system shall display structured error messages to the user in the React UI if a backend API request fails (e.g., utilizing try-catch blocks and Axios interceptors).

[Modified: responsive design framework narrowed from "Bootstrap or Material-UI" to Bootstrap only, per project decision.]

4.2 Software Interfaces (API)

- SMS Gateway (Twilio): The system shall accept HTTP POST requests from Twilio containing inbound SMS data.
- Mock Payment Gateway: The system shall dispatch HTTP POST requests to a simulated financial API to authorize transactions.
- API Testing Interface (Postman): All REST endpoints shall be documented and testable via a shared Postman collection, exposed through environment-specific base URLs.

[Added: Postman as a named software interface, consistent with the new Module 3.6 requirements.]

5. Non-Functional Requirements

5.1 Security

- All backend API routes, except for /auth, /donations/pledge, and /webhook, must be strictly protected by Spring Security utilizing JWT Authorization headers (Bearer <token>).
- Passwords must be hashed using BCrypt before persistence in the MySQL database.
- Cross-Origin Resource Sharing (CORS) must be configured in Spring Boot to only accept requests from the designated React frontend port (and the deployed cloud frontend origin, once hosted).

[Modified: CORS requirement extended to also cover the deployed cloud frontend origin, not just the local dev port.]

5.2 Architecture & Design

- The application must strictly follow the Model-View-Controller (MVC) design pattern.
- The backend architecture must separate concerns into Controller, Service, and Repository layers using @RestController, @Service, and @Repository annotations.

- All API responses must be formatted as structured JSON, standardizing HTTP status codes (200 OK, 400 Bad Request, 403 Forbidden, 404 Not Found, 500 Internal Server Error) using @ControllerAdvice.
- Every error response body shall include, at minimum, the HTTP status code, a human-readable error message, and a timestamp of when the error occurred.

[Added: explicit timestamp requirement for error response bodies, matching the syllabus's error-handling content in Module V.]

5.3 Testing

- All REST API endpoints shall be tested using Postman prior to frontend integration, with test collections covering authentication, CRUD, dispatch workflow, and donation flows.
- Frontend components shall be manually validated against each functional requirement, with results captured as screenshots for the final project report.
- A minimum of one negative test case (invalid input, unauthorized access, or missing resource) shall be documented per module.

[Added new NFR section 5.3 in its entirety: Testing. This directly closes the Postman/testing gap and matches the Capstone Deliverables list ("Test cases and screenshots").]

5.4 Deployment & Reliability

- The application must be completely containerized. A docker-compose.yml file shall orchestrate three interconnected containers (React Frontend, Spring Boot Backend, MySQL Database).
- Database data must be persisted via Docker volumes to prevent data loss upon container restart.
- The containerized application shall be deployed to a cloud platform (e.g., AWS EC2 or Heroku) for final demonstration and evaluation, in addition to local Docker Compose orchestration used during development.
- Source code, Docker configuration, and Postman collections shall be version-controlled in a GitHub repository accessible to the faculty supervisor.

[Renumbered from 5.3 to 5.4 due to insertion of the new Testing section. Added: explicit cloud deployment requirement and GitHub-based version control requirement — both named in the syllabus but absent from the original SRS.]

6. Project Directory Structure

6.1 Repository Structure

The project shall be maintained as a single Git monorepo with the following top-level structure:

```
backend/      # Spring Boot application (Controller-Service-Repository layers)
frontend/    # React.js application
postman/     # API test collections and environment files
docs/        # SRS, ER diagram, architecture diagram, project report
docker-compose.yml
README.md
```

6.2 Backend Structure

Within backend/, code shall be organized by architectural layer, consistent with the MVC and Controller-Service-Repository separation defined in Section 5.2:

```
backend/src/main/java/<base-package>/
  controller/ # REST endpoints
  service/    # Business logic
  repository/ # Data access (Spring Data JPA)
  model/      # Entity classes
  dto/        # Request/response payloads
  security/   # JWT and authentication logic
  config/     # Application and CORS configuration
  exception/  # Centralized error handling

backend/src/main/resources/
  application.properties # MySQL config (dev/prod)

backend/src/test/resources/
  application-test.properties # H2 config (unit/integration tests)
```

[Added: test-resource path for H2 test configuration, separate from the MySQL configuration used in development and production.]

6.3 Frontend Structure

Within frontend/, code shall be organized by responsibility:

```
frontend/src/
  api/          # API call functions
  components/   # Reusable UI elements
  pages/        # Role-based views (Citizen, Volunteer, Admin, Donor)
  context/      # Authentication and shared state
  routes/       # Route protection and navigation
```

6.4 Naming Conventions

All files, folders, packages, and classes shall follow professional naming conventions consistent with standard Java and JavaScript/React practices, and shall be named according to the domain concept they represent (e.g., `SosController`, `InventoryService`, `sosApi.js`) rather than generic or placeholder names. Package and folder names shall be lowercase; Java class names shall use PascalCase; JavaScript/React component files shall use PascalCase, and utility/config files shall use camelCase.

Where development requires the creation of a file, class, or folder not explicitly listed in this structure (e.g., a new controller, service, entity, utility, or configuration class for a future feature), it shall be named following the same conventions defined above and placed within the architectural layer it belongs to (controller/, service/, repository/, model/, dto/, config/, security/, exception/, or the equivalent frontend folder). Names shall be descriptive of the domain concept and responsibility of the file (e.g., `NotificationService.java`, `VolunteerAvailabilityRepository.java`, `donationApi.js`) rather than generic terms such as `Helper.java`, `Utils2.java`, or `NewComponent.jsx`.

[Added new Section 6 in its entirety: Project Directory Structure, covering repository layout, backend and frontend folder organization, and naming conventions for both existing and future files.]